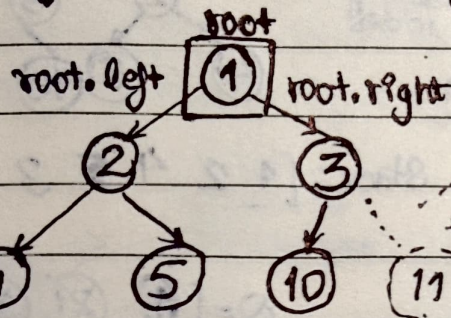


Binary Trees and Binary Search Trees

Max. depth/Height = 3

complete tree →



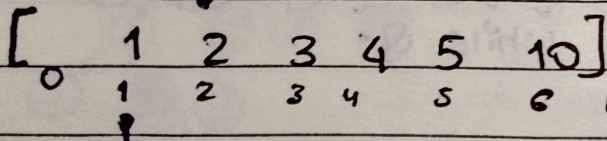
class Node:

root.node = 1

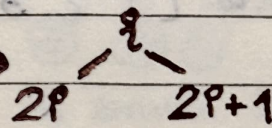
root.left = NONE [some other reference]

root.right =

if this was the case then it would be a perfect tree



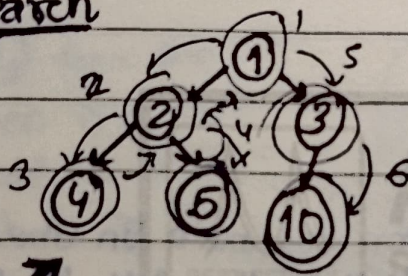
Array Representation



Depth First Search

Implemented using Stack

Recursion



→ Pre-Order Traversal

→ In-Order Traversal

→ Post-Order Traversal

Pre-order

- Node
- Left
- Right

[1 2 4 5 3 10]

Inorder

- Left
- Node
- Right

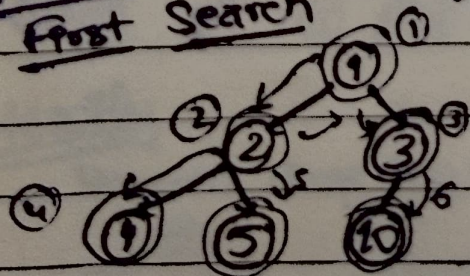
[4 2 5 1 10 3]

Post-order

- Left
- Right
- Node

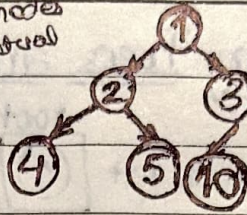
[4 5 2 10 3 1]

Breadth First Search → implemented using Queue



Iterative DFS (no recursion)

we're putting nodes here, not the actual value of the nodes

Stack = [1] ~~2~~ ~~3~~ ~~2~~

while Stack:

~~2~~ ~~3~~ ~~2~~
~~10~~
 [1]

Stack = [1 2 4 5 3 10]

BFS Queue

Q = [1 2 3 4 5 10]

~~Q = [1 2 3 4 5 10]~~
 while Q:

Time: $O(n)$
 Space: $O(n)$

Binary Search Trees

Height Balanced

for any node in the tree, all the stuff to the left is smaller than it and all the stuff to the right is bigger than it.

 lookup: 9
 Time: $O(\log_2 N)$
